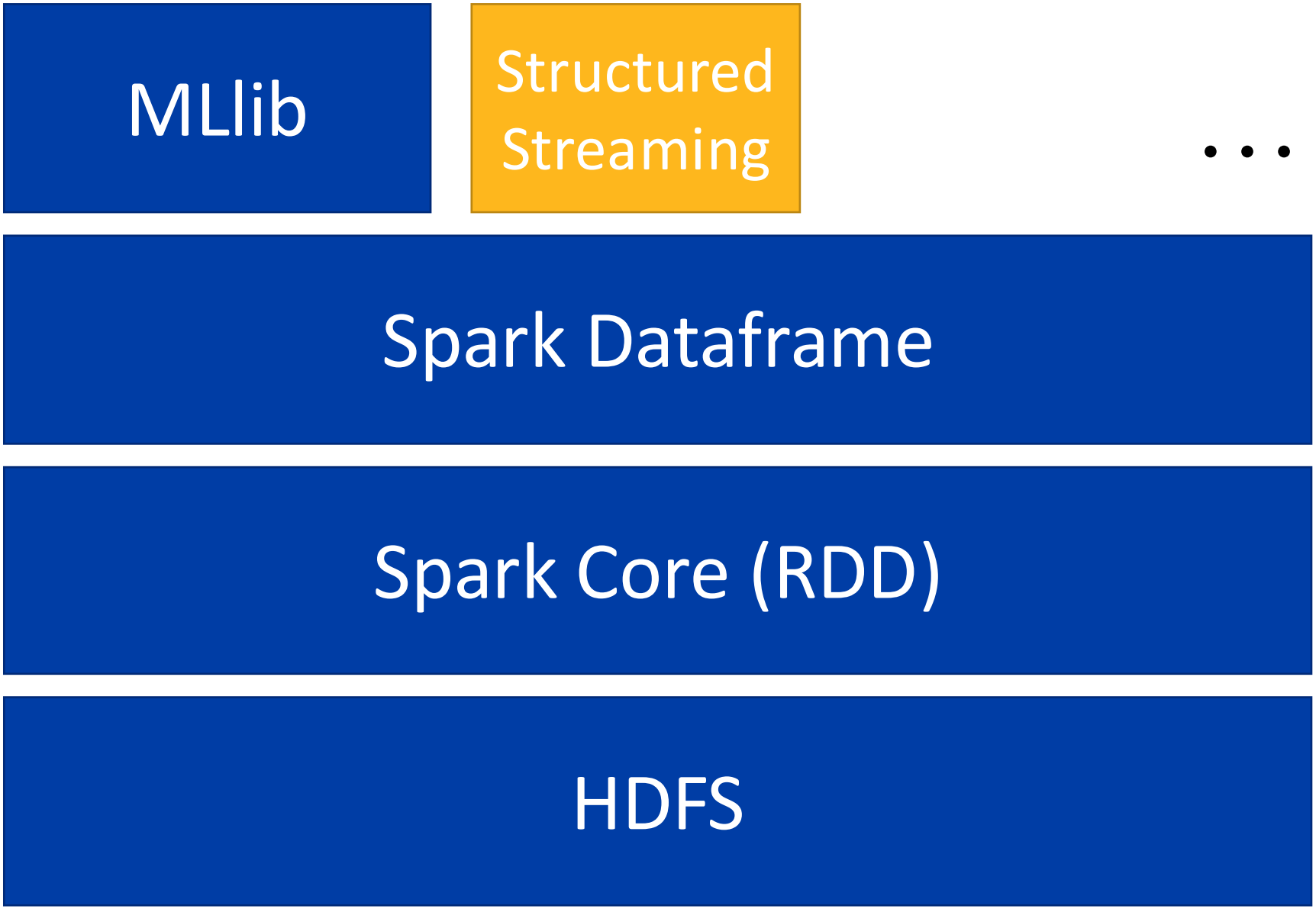
Streaming of Big Data

in Spark

UCR
CELEBRATING 30 YEARS
Marlan and Rosemary Bourns
College of Engineering

Structured Streaming

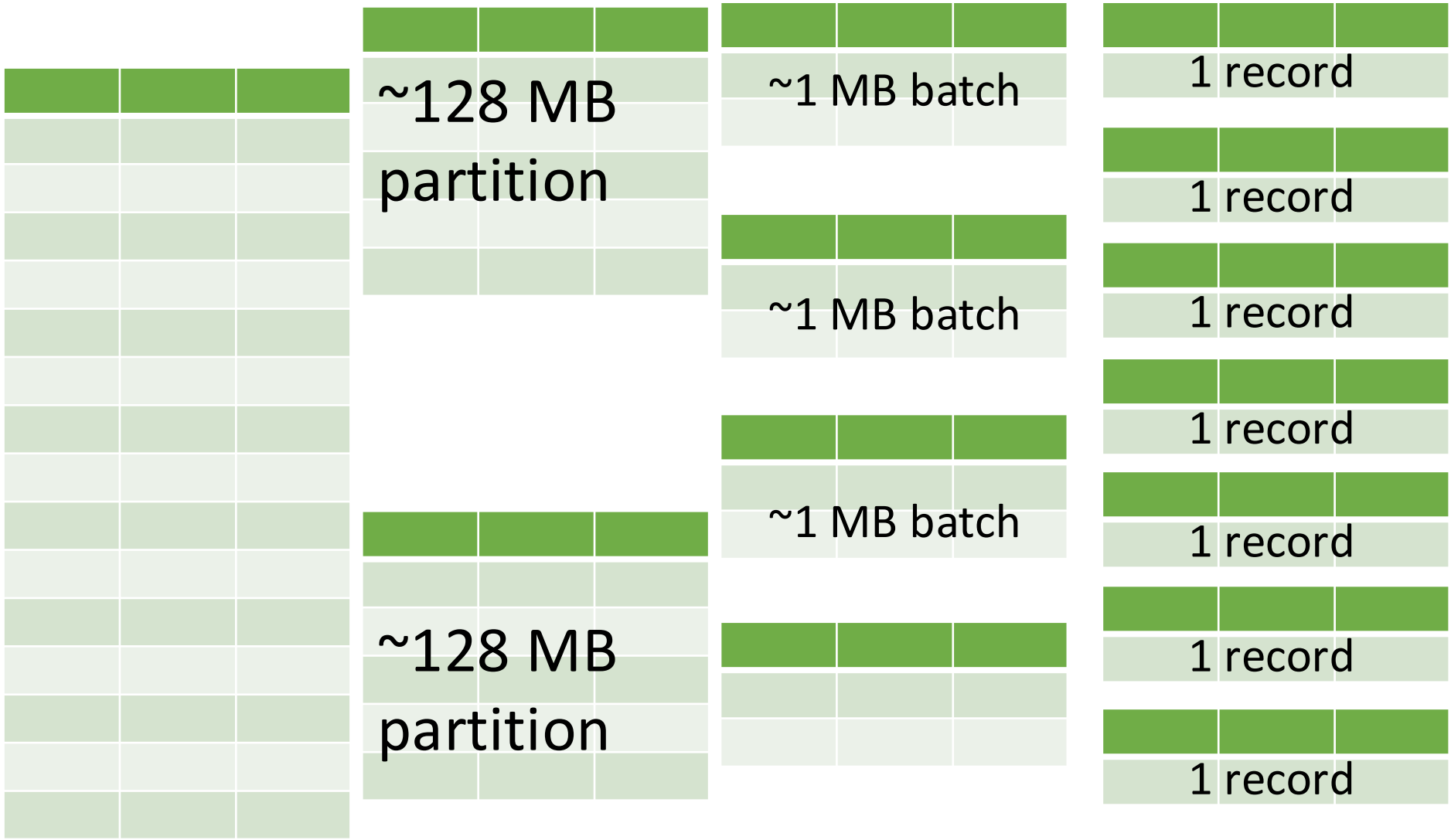


Streaming in Spark

- Spark Streaming
  - Extends the core RDD API to support streaming
  - Not recommended
- Structured Streaming
  - Extends the Dataframe API (SparkSQL)
  - More efficient and recommended

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Types of Streaming



Input Dataframe

Batch processing (SparkSQL)

Micro batching (StructuredStreaming)

Continuous streaming

Word Count Example

```
val spark = SparkSession
  .builder
  .appName("StructuredNetworkWordCount")
  .getOrCreate()

// Create DataFrame from a network connection
val lines = spark.readStream
  .format("socket")
  .option("host", "localhost")
  .option("port", 9999)
  .load()

// Split the lines into words
val words = lines.as[String].flatMap(_.split(" "))

// Generate running word count
val wordCounts = words.groupBy("value").count()

// Start running the query that prints the running counts to the console
val query = wordCounts.writeStream
  .outputMode("complete")
  .format("console")
  .start()

query.awaitTermination()
```

Input/Output

```
apache spark
apache hadoop
```

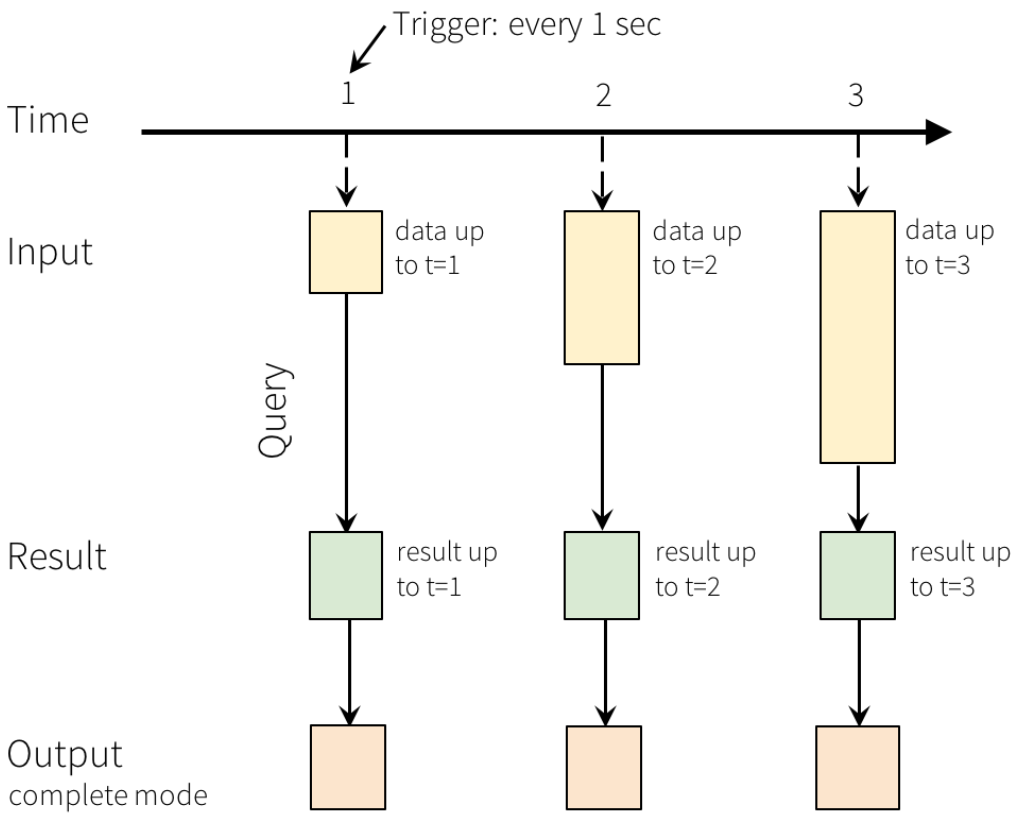
```
-------------------------------------------
Batch: 0
-------------------------------------------
+------+-----+
| value|count|
+------+-----+
|apache|    1|
| spark|    1|
+------+-----+

-------------------------------------------
Batch: 1
-------------------------------------------
+------+-----+
| value|count|
+------+-----+
|apache|    2|
| spark|    1|
|hadoop|    1|
+------+-----+
...
```

Output Modes

- Complete
  - Produce the entire output after each batch
  - Usually useful for debugging and for small output
- Append
  - Only produce new output records
  - No changes are allowed
- Update
  - Produce both new records and modified records

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Complete Output Mode

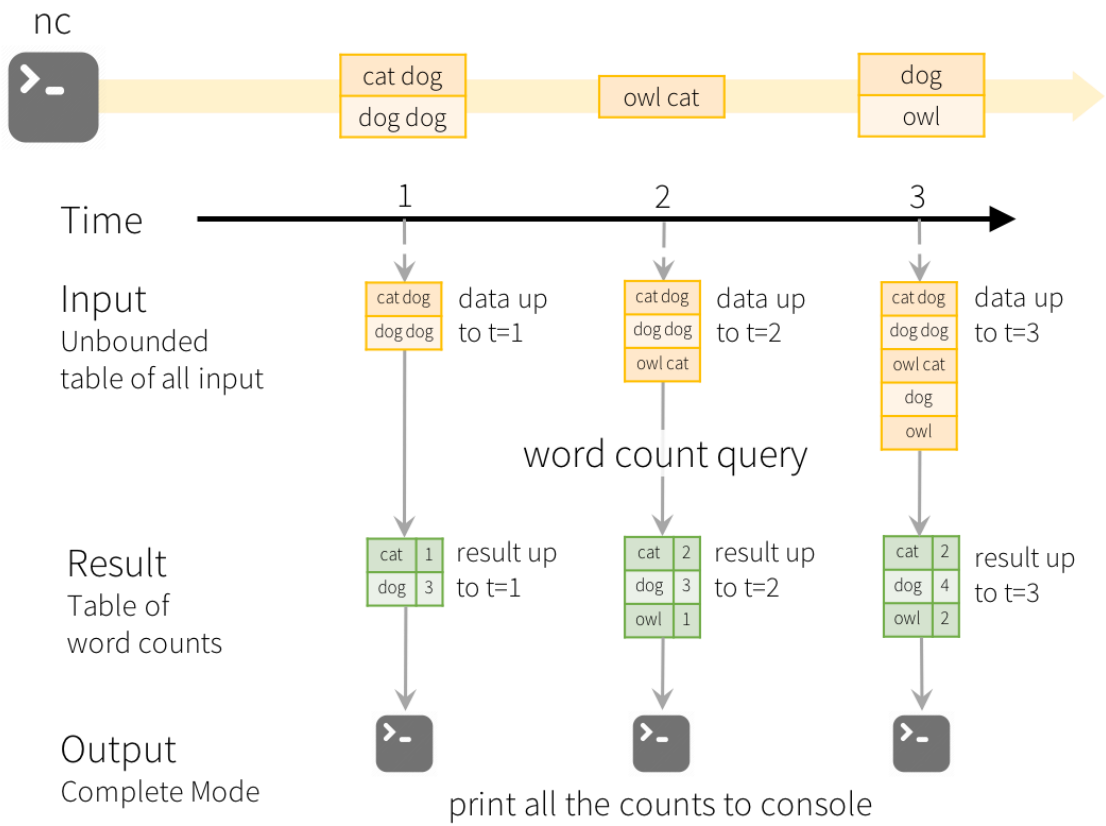


Programming Model for Structured Streaming

Figure: http://spark.apache.org/docs/latest/structured-streaming-programming-guide.html

CELEBRATING 30 YEARS
Marlan and Rosemary Bourns College of Engineering

Complete Output Mode



Model of the Quick Example

Figure: http://spark.apache.org/docs/latest/structured-streaming-programming-guide.html

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Timestamps

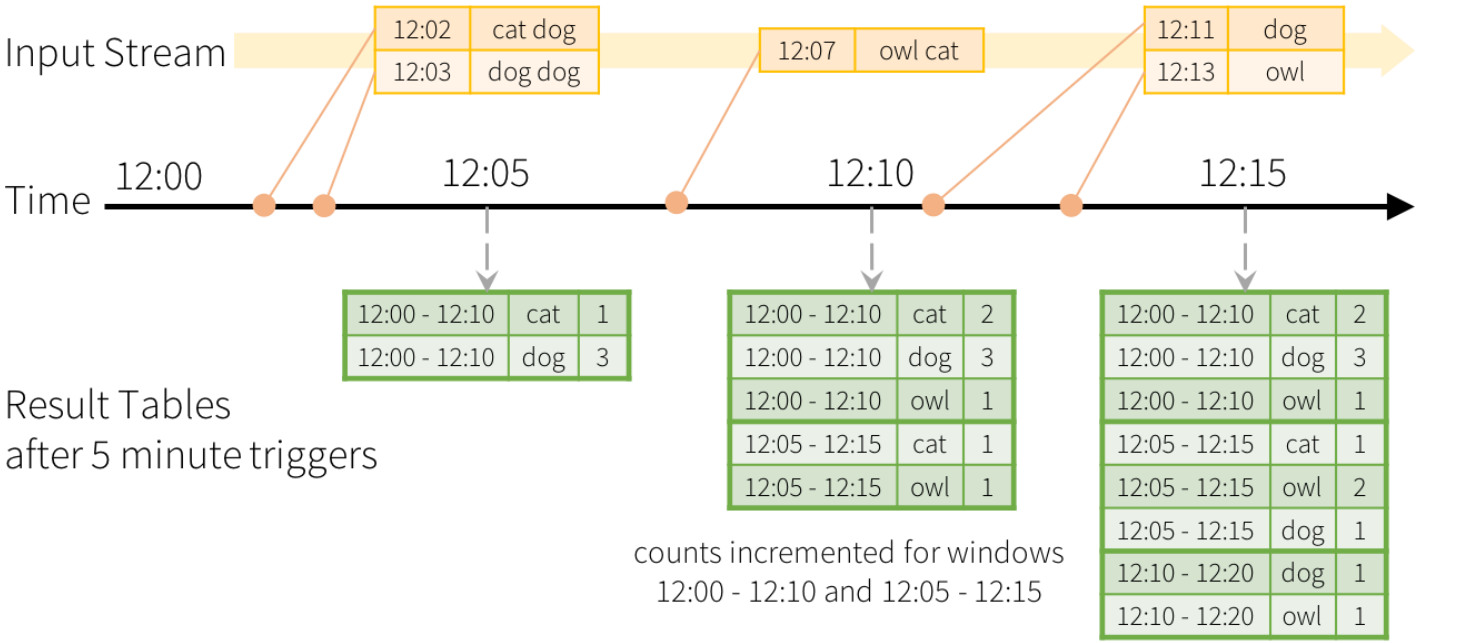
- In some cases, data might come with its own timestamps
  - Weather data
  - Crimes/Incidents
  - Moving objects
  - Tweets
- Structured Streaming allows developers to utilize these timestamps
- The assumption is that the data is already (partially) sorted by this timestamp

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Window Operations

```
// Group the data by window and word and compute the
count of each group
val windowedCounts = words.groupBy(
  window($"timestamp", "10 minutes", "5 minutes"),
  $"word"
).count()
```

Windowed Processing



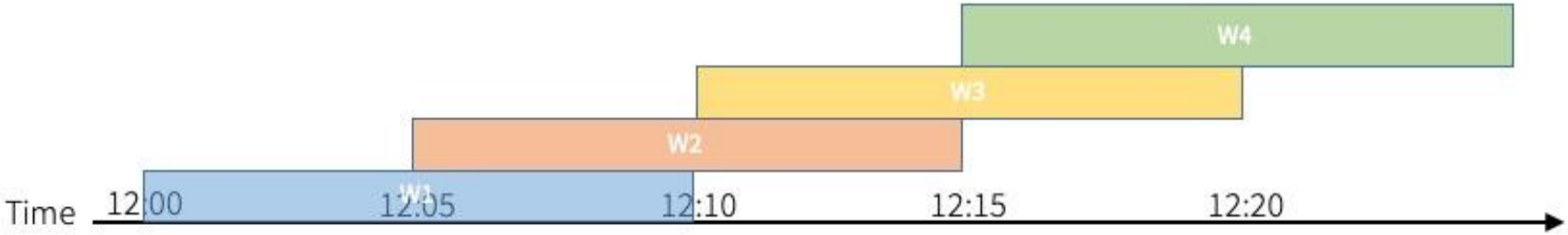
Windowed Grouped Aggregation
with 10 min windows, sliding every 5 mins

UCR CELEBRATING 30 YEARS
Marlan and Rosemary Bourns
College of Engineering

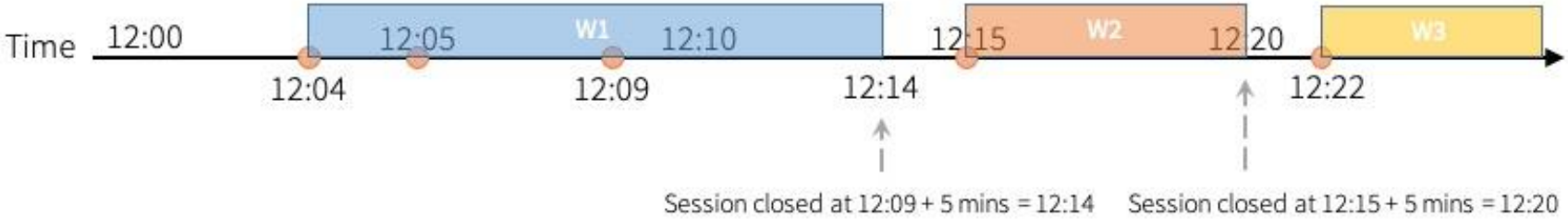
Types of Windows

Tumbling Windows (5 mins)

Sliding Windows (10 mins, slide 5 mins)



Session Windows (gap duration 5 mins)



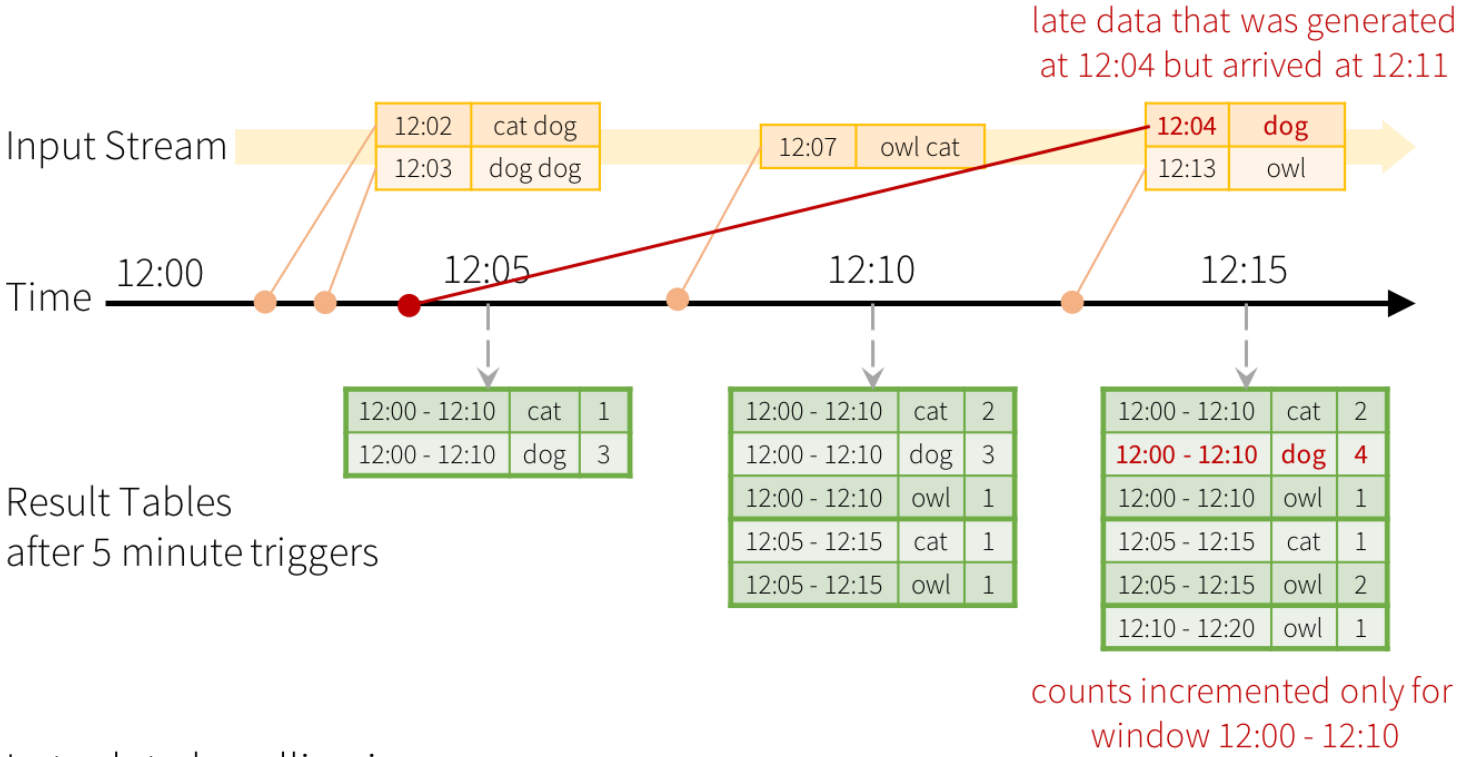
UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Watermarking

- The input data might not be totally ordered
- For example, when merging multiple ordered streams
- Why not ensuring a total ordering?
- Watermarking is a Structured Streaming feature for informing Spark how much error to tolerate

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Watermarking



Late data handling in
Windowed Grouped Aggregation

UCR CELEBRATING 30 YEARS
Marlan and Rosemary Bourns
College of Engineering

Watermarking

```
// Group the data by session window and userId, and compute the
count of each group
val sessionizedCounts = events
    .withWatermark("timestamp", "10 minutes")
    .groupBy(
        session_window($"timestamp", "5 minutes"),
        $"userId")
    .count()
```

Session Windows

```
// Group the data by session window and userId, and compute the
count of each group
val sessionizedCounts = events
    .withWatermark("timestamp", "10 minutes")
    .groupBy(
        session_window($"timestamp", "5 minutes"),
        $"userId")
    .count()
```

Session Windows

```
// Session window with dynamic gap
val sessionWindow = session_window($"timestamp",
     when($"userId" === "user1", "5 seconds")
      .when($"userId" === "user2", "20 seconds")
      .otherwise("5 minutes"))

// Group the data by session window and userId, and compute the
count of each group
val sessionizedCounts = events
    .withWatermark("timestamp", "10 minutes")
    .groupBy(
        Column(sessionWindow),
        $"userId")
    .count()
```

Further Reading

- Official Programming Guide
  - http://spark.apache.org/docs/latest/structured-streaming-programming-guide.html